



SPRING-BOOT 3.5 → 4.0 · MINI GUIDE



Web & HTTP: 3.5 → 4.0

Eight changes at the HTTP layer

One cut from the full Spring Break Cheat Sheets. Every card verified against Spring Boot 3.5.16 and 4.0.7. Real error output, real fixes, from the spring-break test suite. Steve Poole.

About this mini guide

This is one cut from **Spring Break Cheat Sheets**, the field guide to every breaking change between Spring Boot 3.5 and 4.0. Each card covers one change, verified by running real code on 3.5.16 and 4.0.7. One change, one card, one page: the error you'll see, the fix, and the traps.

HOW TO READ A CARD

TIER	How it fails. Tier 1 won't build, Tier 2 won't start, Tier 3 behaves differently. Tier 3 is the dangerous one: nothing goes red, your data just changes shape.
IMPACT	Effort per occurrence once found. S is find-and-replace, M needs an hour of care, L needs a plan.
OPENREWRITE	The badge means the Boot 4.0 recipe fixes this one for you. Run it before touching anything by hand.
WHAT YOU'LL SEE	The real output, so you can match it against your own build.
THE FIX	Before and after code, plus the steps when a rename isn't enough.
WATCH OUT	The traps that make this change look like a different bug.
VERIFY	The command that proves you're done.

THE SERIES

Spring Batch: 3.5 → 4.0	Eight changes between your jobs and 4.0
Hibernate & JPA: 3.5 → 4.0	Ten data-layer changes, mostly the quiet kind
Security & SSO: 3.5 → 4.0	Seven auth changes you cannot ignore
Data & Messaging: 3.5 → 4.0	Nine changes across your stores and streams
Testing: 3.5 → 4.0	Seven test-suite breaks, one clean build away
Observability & Ops: 3.5 → 4.0	Six changes your dashboards will notice first
Jackson & JSON: 3.5 → 4.0	Nine JSON changes everyone hits
Web & HTTP: 3.5 → 4.0	Eight changes at the HTTP layer THIS GUIDE
Core Platform & Build: 3.5 → 4.0	Twenty platform changes underneath everything
The Big Jobs: L-Impact Changes	The six changes that eat your sprint

The full guide collects every change in one book, with a complete planning matrix and the OpenRewrite robot list. github.com/spoole167/spring-break



About HeroDevs

HeroDevs keeps critical open source software secure after the community stops patching it. Never-Ending Support (NES) covers CVEs in Spring Boot 3.5, Java, and dozens of other end-of-life frameworks, so your team migrates on its own schedule instead of the vendor's deadline.

This book exists because migrations fail from surprise more often than difficulty. Every card is a fact, not a guess: verified by running real code on Spring Boot 3.5.16 and 4.0.7.

The complete book, every subsystem in one volume, is coming to LeanPub. Watch github.com/spoole167/spring-break for the link.

herodevs.com

The Planning Matrix

Every change in this book, in book order. Impact is the effort per occurrence once found (S small, M moderate, L large), not how many you have. Auto means the OpenRewrite Boot 4.0 recipe handles it.

8 CHANGES

6 × TIER 1 - WON'T BUILD

2 × TIER 3 - BEHAVES DIFFERENTLY

TIER	CHANGE	IMPACT	AUTO
1	HttpHeaders No Longer Implements MultiValueMap	M	•
1	OkHttp3 Support Removed	M	•
1	RestTemplate Auto-Configuration Removed	S	•
1	Tomcat WAR Deployment Requires New Runtime Starter	S	•
1	Undertow Embedded Server Removed	M	•
1	webjars-locator-core Removed from BOM	S	•
3	DevTools Live Reload Disabled by Default	S	•
3	Path Matching Engine: AntPathMatcher → PathPattern	M	•

The Robot List vs The Human List

The left column is what the OpenRewrite Boot 4.0 recipe fixes while you get coffee. The right column needs a person who understands your code. This split is the honest size of the migration: the renames are free, the behaviour changes are the work.

THE ROBOT LIST

Run the recipe, review the diff, done.

THE HUMAN LIST

Read the card. Check your code. Decide.

HttpHeaders No Longer Implements MultiValueMap

OkHttp3 Support Removed

RestTemplate Auto-Configuration Removed

Tomcat WAR Deployment Requires New Runtime Starter

Undertow Embedded Server Removed

webjars-locator-core Removed from BOM

DevTools Live Reload Disabled by Default

Path Matching Engine: AntPathMatcher → PathPattern

TIER 1

Tier 1 - Won't Build

6 cards

1. HttpHeaders No Longer Implements
MultiValueMap

2. OkHttp3 Support Removed

3. RestTemplate Auto-Configuration Removed

4. Tomcat WAR Deployment Requires New Runtime
Starter

5. Undertow Embedded Server Removed

6. webjars-locator-core Removed from BOM

HttpHeaders No Longer Implements MultiValueMap

Tier 1 - Won't Build

Web & HTTP

Impact: M

WHAT CHANGED

`HttpHeaders` no longer implements `MultiValueMap<String, String>` or `Map<String, List<String>>`. Map-style methods like `containsKey()`, `keySet()`, and `entrySet()` are gone from the type. Header-specific methods replace them.

WHY

Treating HTTP headers as a generic map encouraged misuse and blocked performance work. The dedicated `HttpHeaders` API is more expressive and frees the backing store from the Map contract.

SCOPE CHECK

Search for `MultiValueMap` usages near `HttpHeaders` across your codebase. Also search for `.containsKey()`, `.keySet()`, `.entrySet()` called on variables of type `HttpHeaders`.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/HttpHeadersUsage.java:[8,24]
  error: incompatible types: HttpHeaders cannot be converted to MultiValueMap
[ERROR] /src/main/java/com/example/HttpHeadersUsage.java:[12,16]
  error: cannot find symbol
    symbol: method containsKey(String)
```

THE FIX

Replace Map method calls with HttpHeaders equivalents

BEFORE

```
headers.containsKey("Content-Type")
```

AFTER

```
headers.containsHeader("Content-Type")
```

BEFORE

```
Set<String> names = headers.keySet();
```

AFTER

```
Set<String> names = headers.headerNames();
```

BEFORE

```
Set<Map.Entry<String, List<String>>> entries = headers.entrySet();
```


AFTER

```
Set<Map.Entry<String, List<String>>> entries = headers.headerSet();
```

Passing HttpHeaders where MultiValueMap is expected

BEFORE

```
void process(MultiValueMap<String, String> map) { ... }  
process(headers);
```

AFTER

```
void process(HttpHeaders headers) { ... }  
// or, if the method signature can't change:  
process(headers.toMultiValueMap());
```

HOW TO FIX

Replace Map methods with HttpHeaders methods.

`containsKey(k)` → `containsHeader(k)`, `keySet()` → `headerNames()`, `entrySet()` → `headerSet()`.

These replacements compile on both 3.5 and 4.0.

For method signatures that accept MultiValueMap.

If you have a utility method that accepts `MultiValueMap<String, String>` and you pass `HttpHeaders` into it, either change the parameter type to `HttpHeaders`, or call `headers.toMultiValueMap()` at the call site.

WATCH OUT

Code in Spring interceptors, filters, and custom converters that inspects headers using Map idioms is the most common source of these failures. Check

`HandlerInterceptor`,
`ClientHttpRequestInterceptor`, and
`ResponseBodyAdvice` implementations.

`toMultiValueMap()` returns a copy, not a live view. Mutations to the returned map do not affect the original `HttpHeaders`.

FURTHER INFO

Driven by Spring Framework 7.0. HttpHeaders had been a MultiValueMap subtype since early in Spring's history.

spring-break module: `httpheaders-multivaluemap` Spring Framework 7.0 Release Notes

OkHttp3 Support Removed

Tier 1 - Won't Build

Web & HTTP

Impact: M

Spring Boot 4.0 dropped the OkHttp3 client factory and BOM entry. Projects using OkHttp3RestClient or the managed dependency won't compile.

WHAT CHANGED

The `OkHttp3ClientHttpRequestFactory` class was removed from Spring Framework 7.0. Spring Boot 4.0 also removed `com.squareup.okhttp3` from its managed dependency BOM. Any code creating a `RestClient` or `RestTemplate` with the OkHttp3 factory won't compile.

WHY

The JDK's built-in `java.net.http.HttpClient` (since Java 11) and the Apache `HttpClient` 5 cover the same use cases. Maintaining a third HTTP client integration added complexity without clear benefit.

SCOPE CHECK

Search for `OkHttp3` and `com.squareup.okhttp3` across your codebase and build files. Check both production code and test configurations: OkHttp's `MockWebServer` is also affected.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/config/RestConfig.java:[5,48]
package org.springframework.http.client.OkHttp3ClientHttpRequestFactory does not exist
[ERROR] /src/main/java/com/example/config/RestConfig.java:[14,16]
cannot find symbol
  symbol:   class OkHttp3ClientHttpRequestFactory
  location: class com.example.config.RestConfig
```

THE FIX

RestClient configuration

BEFORE

```
import org.springframework.http.client.OkHttp3ClientHttpRequestFactory;

@Bean
public RestClient restClient() {
    return RestClient.builder()
        .requestFactory(new OkHttp3ClientHttpRequestFactory())
        .build();
}
```

AFTER

```
import org.springframework.http.client.JdkClientHttpRequestFactory;

@Bean
public RestClient restClient() {
    return RestClient.builder()
        .requestFactory(new JdkClientHttpRequestFactory())
        .build();
}
```

pom.xml — remove OkHttp dependency

BEFORE

```
<dependency>
  <groupId>com.squareup.okhttp3</groupId>
  <artifactId>okhttp</artifactId>
</dependency>
```

AFTER

```
<!-- JdkClientHttpRequestFactory uses java.net.http – no extra dependency needed -->
```

HOW TO FIX

Switch to JdkClientHttpRequestFactory (simplest).

Replace `OkHttp3ClientHttpRequestFactory` with `JdkClientHttpRequestFactory`. It uses the JDK's built-in HTTP client and requires no additional dependencies.

Switch to Apache HttpClient 5.

If you need connection pooling or advanced proxy configuration, use `HttpComponentsClientHttpRequestFactory` with Apache HttpClient 5. Add `org.apache.httpcomponents.client5:httpclient5` to your POM.

WATCH OUT

OkHttp interceptors for logging, retries, or authentication have no direct equivalent in the JDK client. Reimplement that logic with Spring's `ClientHttpRequestInterceptor`.

OkHttp's `MockWebServer` is popular in tests. If you only use OkHttp for testing, consider switching to `MockRestServiceServer` or WireMock instead.

FURTHER INFO

OkHttp itself still works as a plain library; what vanished is Spring's factory integration and the managed BOM version.

[Spring-Break Demo](#) [Spring Boot 4.0 Migration Guide](#)

RestTemplate Auto-Configuration Removed

Tier 1 - Won't Build

Web & HTTP

Impact: S

WHAT CHANGED

Spring Boot 4.0 removed the `org.springframework.boot.web.client` package. `RestTemplateBuilder` and its support classes relocated to the new `spring-boot-restclient` module, which covers both `RestClient` and `RestTemplate`.

WHY

`RestTemplate` has been in maintenance mode since Spring 5, and Spring Framework 6.1 introduced `RestClient` as its fluent replacement. Removing the auto-configuration pushes migration to `RestClient` or `WebClient`.

SCOPE CHECK

Search for `RestTemplate`, `RestTemplateBuilder`, and `RestTemplateCustomizer` across your codebase. Every import from the old package is a compile failure. Also check test classes that use `TestRestTemplate`: its auto-configuration may also be affected.

WHAT YOU'LL SEE

```
$ mvn clean compile
[ERROR] COMPILATION ERROR :
[ERROR] OrderClient.java:[3,42] package
    org.springframework.boot.web.client does not exist
[ERROR] OrderClient.java:[12,25] cannot find symbol
    symbol: class RestTemplateBuilder
[INFO] BUILD FAILURE
---
The build stops at compile. The application never starts.
```

THE FIX

Injecting the HTTP client — service class

BEFORE

```
@Service
public class OrderClient {
    private final RestTemplate restTemplate;

    public OrderClient(RestTemplateBuilder builder) {
        this.restTemplate = builder
            .rootUri("https://api.example.com")
            .build();
    }

    public Order getOrder(Long id) {
        return restTemplate.getForObject(
            "/orders/{id}", Order.class, id);
    }
}
```

AFTER

```
@Service
public class OrderClient {
    private final RestClient restClient;

    public OrderClient(RestClient.Builder builder) {
        this.restClient = builder
            .baseUrl("https://api.example.com")
            .build();
    }

    public Order getOrder(Long id) {
        return restClient.get()
            .uri("/orders/{id}", id)
            .retrieve()
            .body(Order.class);
    }
}
```

Test — using @RestClientTest

BEFORE

```
@RestClientTest(OrderClient.class)
class OrderClientTest {
    @Autowired
    private MockRestServiceServer server;
    @Autowired
    private OrderClient client;
```

AFTER

```
@RestClientTest(OrderClient.class)
class OrderClientTest {
    @Autowired
    private MockRestServiceServer server;
    @Autowired
    private OrderClient client;
    // Works the same – @RestClientTest supports RestClient
```

HOW TO FIX

Migrate to RestClient (recommended).

Replace `RestTemplate` usage with `RestClient`. Inject `RestClient.Builder` (auto-configured by Spring Boot 4) instead of `RestTemplateBuilder`. The API is fluent and supports the same customisers.

Update the RestTemplateBuilder import.

If migration is too large to do now, keep `RestTemplate`: add the `spring-boot-restclient` module (or its starter) to your build and update the `RestTemplateBuilder` import. This restores the old behaviour while you plan the migration.

Use WebClient for reactive code.

Reactive applications should use `WebClient`, which remains auto-configured in Spring Boot 4.0.

WATCH OUT

`TestRestTemplate` is a separate concern. Check whether it is still auto-configured in your test slices or if you need to provide it manually.

The `RestClient` API is different from `RestTemplate`. Methods like `getForObject()` and `postForEntity()` become chained `.get().uri().retrieve().body()` calls. Budget time for the API rewrite as well as the bean swap.

Third-party libraries that inject `RestTemplateBuilder` (e.g., Spring Cloud OpenFeign, some tracing libraries) may also break. Check transitive dependencies.

FURTHER INFO

Part of Spring Boot 4.0's wider modularisation of HTTP client support. See also: [okhttp3-removed](#).

[Spring-Break Demo](#) [Testing changes in Spring Boot 4.0](#) [Spring Boot 4.0 Migration Guide](#)

Tomcat WAR Deployment Requires New Runtime Starter

Tier 1 - Won't Build

Web & HTTP

Impact: S

Boot 4.0 adds `spring-boot-starter-tomcat-runtime` for WAR deployment. The old provided-scope `spring-boot-starter-tomcat` pattern may conflict with the external container's classpath.

WHAT CHANGED

A new `spring-boot-starter-tomcat-runtime` artifact covers WAR deployment. It provides the runtime bridge classes needed to run a Boot application inside an external Tomcat, without the full embedded Tomcat server dependencies. `spring-boot-starter-tomcat` is now exclusively for embedded use.

WHY

Embedded Tomcat and external container are distinct deployment models with different runtime dependencies. The old trick of marking `spring-boot-starter-tomcat` as `provided` blurred that line and was error-prone.

SCOPE CHECK

Search `pom.xml` for `spring-boot-starter-tomcat` with `scope=provided`. Also check `SpringBootServletInitializer` subclasses: any WAR project using that class is affected.

WHAT YOU'LL SEE

```
// Deploying WAR to external Tomcat with Boot 3.5 dependency pattern:
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-tomcat</artifactId>
  <scope>provided</scope>
</dependency>

// Boot 4.0: may result in:
java.lang.ClassNotFoundException: jakarta.servlet.FilterRegistration
// Or: version conflicts between embedded and container Tomcat classes
```

THE FIX

pom.xml — WAR deployment dependency update

BEFORE

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-tomcat</artifactId>
  <scope>provided</scope>
</dependency>
```

AFTER

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-tomcat-runtime</artifactId>
  <scope>provided</scope>
</dependency>
```

HOW TO FIX

Replace `spring-boot-starter-tomcat` (provided) with `spring-boot-starter-tomcat-runtime`.

In your WAR project's `pom.xml`, replace the provided-scope `spring-boot-starter-tomcat` dependency with `spring-boot-starter-tomcat-runtime`. Keep the `provided` scope: the external container supplies Tomcat at runtime.

VERIFY

WAR file deploys to external Tomcat without `ClassNotFoundException` or dependency conflicts after swapping to `spring-boot-starter-tomcat-runtime`

WATCH OUT

If your project also runs as an executable JAR (dual deployment mode), keep `spring-boot-starter-tomcat` for the embedded server but add `spring-boot-starter-tomcat-runtime` as provided.

FURTHER INFO

The Boot 3.5 pattern, `spring-boot-starter-tomcat` at `provided` scope, can still work on 4.0. Mixing it with the new split risks classpath conflicts or `ClassNotFoundException` at deployment time.

Spring Boot 4.0 Migration Guide

Undertow Embedded Server Removed

Tier 1 - Won't Build

Web & HTTP

Impact: M

WHAT CHANGED

The `spring-boot-starter-undertow` module was removed from Spring Boot 4.0. Undertow is no longer a supported embedded servlet container. The supported options are Tomcat (default), Jetty, and Netty (for reactive).

WHY

Undertow's development slowed and its Jakarta EE 11 support lagged behind Tomcat and Jetty. Maintaining a third servlet container integration stopped being worth the support burden.

SCOPE CHECK

Search for `spring-boot-starter-undertow` in POM and Gradle files. Also search for `server.undertow` in application properties/YAML files: those properties won't cause a build failure but will be silently ignored.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] Failed to execute goal on project my-service:
Could not resolve dependencies for project com.example:my-service:jar:1.0.0:
The following artifacts could not be resolved:
org.springframework.boot:spring-boot-starter-undertow:jar:4.0.0 (not found)
```

THE FIX

pom.xml — remove Undertow starter

BEFORE

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
  </exclusions>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-undertow</artifactId>
</dependency>
```

AFTER

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<!-- Uses Tomcat by default. For Jetty, add spring-boot-starter-jetty
and exclude spring-boot-starter-tomcat -->
```

BEFORE

```
server:
  undertow:
    io-threads: 8
    worker-threads: 64
    buffer-size: 1024
```

AFTER

```
server:
  tomcat:
    threads:
      max: 200
      min-spare: 10
```

HOW TO FIX

Switch to Tomcat (default).

Remove the `spring-boot-starter-undertow` dependency and the Tomcat exclusion. Tomcat is the default: `spring-boot-starter-web` alone is enough.

Switch to Jetty.

If you need a non-Tomcat server, use `spring-boot-starter-jetty` instead. Exclude the Tomcat starter and add the Jetty starter.

VERIFY

App starts on Tomcat/Jetty with no Undertow class errors

WATCH OUT

Undertow-specific configuration properties under `server.undertow.*` have no Tomcat or Jetty equivalent. Review your `application.yml` and translate settings like `io-threads`, `worker-threads`, and `buffer-size` to the new server's configuration model.

If you chose Undertow for its non-blocking I/O on the servlet side, consider switching to Spring WebFlux with Netty instead of forcing the same architecture onto Tomcat.

FURTHER INFO

One of the few 4.0 breaks with no like-for-like replacement: you must change servers.

spring-break module: undertow-removed [Spring Boot 4.0 Migration Guide](#)

webjars-locator-core Removed from BOM

Tier 1 - Won't Build

Web & HTTP

Impact: S

Spring Boot 4.0 removes `webjars-locator-core` from the BOM. Version-agnostic WebJar URLs (`/webjars/jquery/jquery.min.js`) stop resolving. Switch to `webjars-locator-lite`.

WHAT CHANGED

`org.webjars:webjars-locator-core` was the original WebJar asset locator, using full classpath scanning to find versioned WebJar paths. Spring Boot 4.0 removes it from the BOM and its auto-configuration wiring, replacing it with `org.webjars:webjars-locator-lite`, which uses a pre-generated manifest file (`webjars-requirejs.js` or `META-INF/resources/webjars/`) to locate assets without scanning.

WHY

The classpath scanning approach in `webjars-locator-core` added startup overhead proportional to the number of JARs on the classpath and was incompatible with GraalVM native image compilation. The lite variant eliminates both problems: manifest-based lookup is O(1) and works in native images.

SCOPE CHECK

Search for `webjars-locator-core` in `pom.xml` and `build.gradle` files. Also grep your Java source for `import org.webjars.WebJarAssetLocator`. Check Thymeleaf templates and JSPs for version-agnostic WebJar URLs (`th:href="@{/webjars/bootstrap/css/bootstrap.min.css}"`): these require a locator to resolve and will 404 if no locator is present.

WHAT YOU'LL SEE

```
// pom.xml — dependency with no version (relies on BOM)
<dependency>
  <groupId>org.webjars</groupId>
  <artifactId>webjars-locator-core</artifactId>
</dependency>

// Boot 3.5: BOM provides version 0.59 — resolves fine.

// Boot 4.0: BOM no longer manages webjars-locator-core.
[ERROR] 'dependencies.dependency.version' for
org.webjars:webjars-locator-core:jar is missing.
[ERROR] BUILD FAILURE

// Or if version is pinned in pom.xml — compiles but WebJarAssetLocator
// is no longer registered by Boot's auto-configuration:
java.lang.ClassNotFoundException: org.webjars.WebJarAssetLocator
```

THE FIX

pom.xml — replace locator dependency

BEFORE

```
<dependency>
  <groupId>org.webjars</groupId>
  <artifactId>webjars-locator-core</artifactId>
</dependency>
```

AFTER

```
<dependency>
  <groupId>org.webjars</groupId>
  <artifactId>webjars-locator-lite</artifactId>
</dependency>
```

Code using WebJarAssetLocator directly

BEFORE

```
import org.webjars.WebJarAssetLocator;
WebJarAssetLocator locator = new WebJarAssetLocator();
String path = locator.getFullPath("jquery", "jquery.min.js");
```

AFTER

```
import org.webjars.lite.WebJarAssetLocator;
WebJarAssetLocator locator = new WebJarAssetLocator();
String path = locator.getFullPath("jquery", "jquery.min.js");
```

HOW TO FIX

Replace webjars-locator-core with webjars-locator-lite.

Update `pom.xml` or `build.gradle` to declare `org.webjars:webjars-locator-lite`. The API is compatible for common use cases: Spring MVC's WebJar resource handler works with both.

Update direct WebJarAssetLocator imports.

If your code imports `org.webjars.WebJarAssetLocator` directly, change the import to the `org.webjars.lite` package. The method signatures for `getFullPath()` are compatible.

VERIFY

WebJar assets resolve correctly via the webjars-locator-lite replacement

WATCH OUT

If you pin an explicit version of `webjars-locator-core` in your pom, the build succeeds, but Boot 4.0 no longer wires the auto-configuration. WebJar URLs that worked without a version segment silently return 404 instead of failing at startup.

`webjars-locator-lite` requires that WebJar dependencies include a `webjars-requirejs.js` manifest. Older WebJar artifacts may not include this file. Check the WebJar version you are using is compatible with the lite locator.

FURTHER INFO

Both locators come from the WebJars project itself; Spring Boot switched which one it manages and wires.

spring-break module: `webjars-locator-core-removed`

[Spring Boot 4.0 Migration Guide](#)

[webjars-locator-lite on GitHub](#)

TIER 3

Tier 3 - Behaves Differently

2 cards

1. DevTools Live Reload Disabled by Default

2. Path Matching Engine: AntPathMatcher → PathPattern

DevTools Live Reload Disabled by Default

Tier 3 - Behaves Differently

Web & HTTP

Impact: S

Boot 4.0 disables DevTools Live Reload by default: no automatic browser refresh after code changes. Set

`spring.devtools.livereload.enabled=true` in your dev profile to restore it.

WHAT CHANGED

The `spring.devtools.livereload.enabled` property now defaults to `false`. The Live Reload server is not started unless this property is explicitly set to `true`.

WHY

Live Reload opens a port (35729) on the developer's machine and needs a browser extension. Starting it unasked surprised developers who did not know about or want it. Opt-in gives explicit control over which DevTools features are active.

SCOPE CHECK

Check `application.properties` and `application-dev.properties` for any existing DevTools configuration.

WHAT YOU'LL SEE

```
# Boot 3.5: live reload works automatically with devtools on classpath.
# Boot 4.0: live reload disabled. Browser does not refresh.
# No error: the server restarts but the browser stays on the old page.
```

THE FIX

`application-dev.properties` or `application.properties`

BEFORE

```
# Boot 3.5: live reload on by default, no property needed
```

AFTER

```
# Boot 4.0: must opt in
spring.devtools.livereload.enabled=true
```

HOW TO FIX

Add `spring.devtools.livereload.enabled=true` to your dev profile.

Add the property to `application-dev.properties` or your development-time configuration. Keep it out of production profiles: DevTools should not be on the production classpath anyway, and scoping the property to dev makes the intent clear.

VERIFY

Browser refreshes automatically after code changes when `spring.devtools.livereload.enabled=true` is set in development profile

WATCH OUT

The DevTools automatic restart (triggered by classpath changes) is separate from Live Reload and unaffected. The app still restarts; the browser just does not receive the push notification to refresh.

FURTHER INFO

In Boot 3.5, adding `spring-boot-devtools` to the classpath automatically started a Live Reload server on port 35729, which pushed browser refresh signals after application restarts.

[Spring Boot 4.0 Migration Guide](#) [Spring Boot DevTools Documentation](#)

Path Matching Engine: AntPathMatcher → PathPattern

Tier 3 - Behaves Differently

Web & HTTP

Impact: M

Spring Security now uses `PathPattern` instead of `AntPathMatcher`. Some wildcard patterns match differently, causing silent authorisation gaps or false denials.

WHAT CHANGED

Spring Boot 4.0 switched the default URL path matching engine from `AntPathMatcher` to `PathPatternParser` for both MVC request mapping and Security request matchers. While most patterns work identically, edge cases around `**` in the middle of patterns, trailing slashes, and encoded path segments differ.

WHY

`PathPatternParser` is significantly faster (pre-parsed at startup, no runtime string manipulation), handles path variables and matrix parameters correctly, and is the default in Spring WebFlux. Unifying on one engine simplifies the framework.

SCOPE CHECK

Check every `.requestMatchers()` call in your security configuration and every `@RequestMapping` pattern in your controllers. Patterns with **in the middle, trailing slashes, or regex segments are the most likely to break**. Simple patterns like `/api/users/` (at the end) are generally safe.

WHAT YOU'LL SEE

```
// Security configuration – unchanged
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/**/admin").hasRole("ADMIN")
);

// Before (AntPathMatcher – Spring Boot 3.5)
GET /api/v1/admin          → 403 (matched, requires ADMIN)
GET /api/v1/users/admin    → 403 (matched, ** crosses segments)

// After (PathPattern – Spring Boot 4.0)
GET /api/v1/admin          → 403 (still matched)
GET /api/v1/users/admin    → 200 (NOT matched – pattern behaves differently)

// Security test failure
Expected: request denied (403)
Actual: request allowed (200)
// Potential security hole: endpoint exposed without authorisation
```

THE FIX

Fix: use `PathPattern` syntax for multi-segment match

BEFORE

```
.requestMatchers("/api/**/admin").hasRole("ADMIN")
```

AFTER

```
.requestMatchers("/api/{*path}/admin").hasRole("ADMIN")
```


Trailing slash no longer matches by default

BEFORE

```
.requestMatchers("/api/users").authenticated()
```

AFTER

```
.requestMatchers("/api/users", "/api/users/").authenticated()
```

Explicit matcher construction: `AntPathRequestMatcher` is gone

BEFORE

```
.requestMatchers(new AntPathRequestMatcher("/public/**"))  
.permitAll()
```

AFTER

```
.requestMatchers(PathPatternRequestMatcher.withDefaults()  
.matcher("/public/**")).permitAll()
```

HOW TO FIX

Audit all path patterns in security config.

Review every `requestMatchers()` pattern for uses of `**` in the middle (not at the end), trailing slashes, and URL-encoded segments. Test each pattern with PathPattern semantics. Use `{*varName}` for capturing multi-segment path variables.

Rewrite every legacy pattern: there is no fallback.

Spring Security 7 removes `AntPathRequestMatcher` entirely, so you cannot keep the old engine for individual rules. Explicit matcher construction uses `PathPatternRequestMatcher.withDefaults().matcher(pattern)`, and the pattern itself must be valid PathPattern syntax.

VERIFY

All API endpoints respond (especially wildcard/regex patterns)

WATCH OUT

A pattern that no longer matches is a security hole: paths that `/api/**/admin` used to protect become unprotected. A Tier 3 change with Tier 0 security implications.

`PathPattern` does not support `**` in the middle of a pattern (e.g. `/api/ /admin`). It only supports `**` at the end (e.g. `/api/`). Use `{*path}` for variable-length segment capture in the middle.

FURTHER INFO

`PathPatternParser` has been WebFlux's default since Spring 5.0; Boot 4.0 brings MVC and Security into line. See also: [security-dsl-rewrite](#), [auth-default-deny](#).

[Spring-Break Demo](#) [Spring Security 7 migration guide](#) [Spring Boot 4.0 Migration Guide](#)



Not ready to migrate?

Spring Boot 3.5 stops receiving fixes when open-source support ends. You have three sane options. Pick one deliberately.

1. **Pin 3.5 and track CVEs.** Free, but the clock is running and every new CVE is your team's problem to triage.
2. **Migrate incrementally with OpenRewrite.** The Boot 4.0 recipe automates the renames in this book; the behavioural changes still need eyes.
3. **HeroDevs NES for Spring.** Security patches for 3.5 beyond EOL, buying your team time to migrate on your schedule, not the calendar's.

Spring Boot 4.0 Migration Guide • github.com/spring-projects/spring-boot/wiki

OpenRewrite Boot 4.0 recipe • docs.openrewrite.org/recipes/java/spring/boot4

HeroDevs NES for Spring • herodevs.com/support/spring-nes

The test suite behind this book • github.com/spoole167/spring-break